Guía visual y técnica de Cancha Clara

Esta guía acompaña el Avance 1 y permite recorrer el código, interpretar las imágenes y repetir las pruebas. Las cuatro pantallas siguientes son bocetos del producto previsto. La captura de Angular corresponde a la pantalla inicial que sí está implementada.

Cómo ejecutar el proyecto

```
cd prototipo-ts
npm install
npm run build
npm start
npm test
```

npm install descarga las dependencias. npm run build revisa los tipos y genera dist. npm start ejecuta la demostración. npm test compila y ejecuta los seis casos de prueba; todos deben finalizar sin fallos.

```
cd ../frontend
npm install
npm start
```

Abrir http://localhost:4200. Para detener el servidor, usar Ctrl+C en esa terminal. Para generar la compilación de Angular, ejecutar npm run build desde frontend/. Se utilizó Node.js 24.14.0, TypeScript 5.9.3 en el prototipo y Angular CLI 20.3.42.

Dónde está cada responsabilidad

Archivo o carpeta	Qué conviene revisar
src/models/	Las cuatro interfaces, sus id readonly y las uniones de literales.
src/utils/reglas.ts	Condiciones de aceptación y rechazo de cada operación.
src/utils/repositorio.ts	Almacenamiento temporal y búsqueda con resultado opcional.
src/utils/tarifas.ts	Jerarquía que calcula las cotizaciones.
src/datos.ts y main.ts	Datos ficticios y recorrido de demostración.
frontend/src/app/	Modelos copiados, componentes, HTML y estilos.

Inicio y resumen de la jornada

Boceto 01 Inicio y resumen de la jornada



Navegación: «Consultar canchas» abre el boceto 02. El menú permite volver al inicio.

Pantalla prevista para un avance posterior. Los valores son datos de ejemplo.

Boceto elaborado para este proyecto. Las notas inferiores indican navegación y condiciones de uso.

Qué muestra y por qué

El resumen permite a recepción ubicarse antes de registrar una reserva. Las cifras del boceto son coherentes con los datos de ejemplo: cuatro canchas activas, cuatro reservas el 6 de noviembre y 17 000 CRC pendientes ese día. La reserva R05 pertenece al día siguiente y no entra en ese saldo.

Cómo se usaría

El botón de consulta lleva a la lista de canchas. La navegación superior permite cambiar de área. En este avance Angular solo muestra una presentación del proyecto y una cancha; estos indicadores todavía son parte del diseño.

Requerimientos relacionados: RF-03, RF-08 y RF-10.

Lista de canchas

Boceto 02 Lista de canchas



Navegación: «Ver detalle» abre el boceto 03 de la cancha elegida. «Inicio» regresa al 01.

El filtro excluye las canchas en mantenimiento; corresponde a la regla C1.

Boceto elaborado para este proyecto. Las notas inferiores indican navegación y condiciones de uso.

Qué muestra y por qué

Cada fila responde tres preguntas: qué cancha es, cuánto cuesta por hora y dónde consultar su detalle. El filtro por deporte reduce las opciones. La cancha en mantenimiento no se ofrece como reservable, aunque se conserva en los datos.

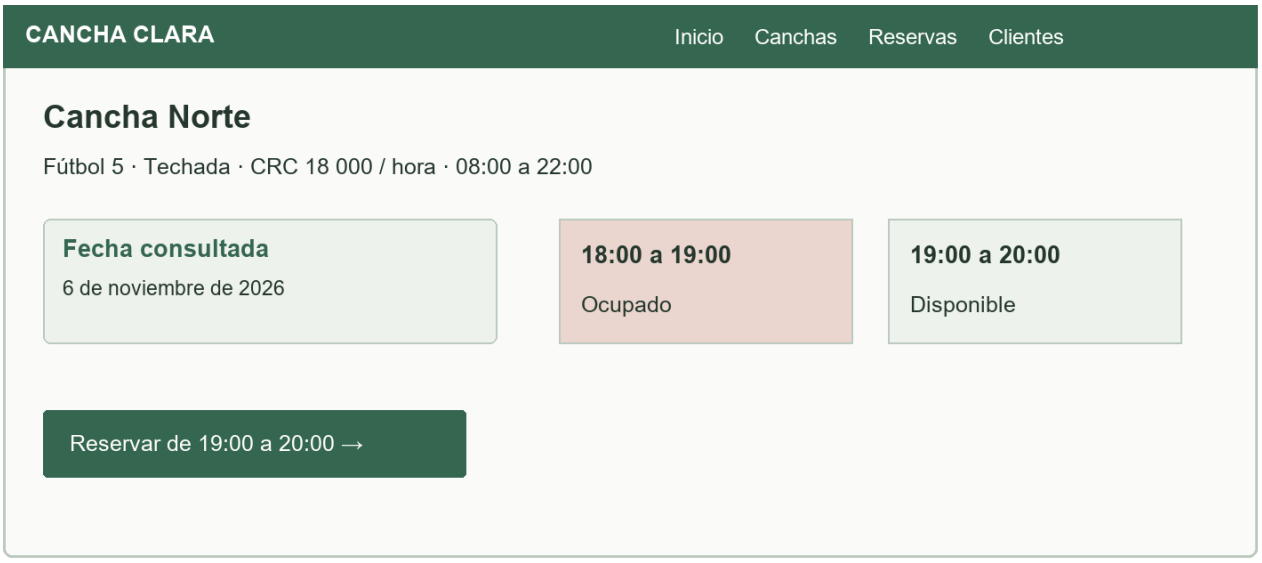
Cómo se usaría

Al pulsar Ver detalle se abre la cancha seleccionada. El filtro de este boceto corresponde a canchasActivas, que usa filter para incluir solo canchas activas del deporte indicado.

Requerimientos relacionados: RF-01 y RF-03.

Detalle y disponibilidad

Boceto 03 Detalle de cancha y disponibilidad



Navegación: «Reservar» abre el boceto 04 con la cancha y la hora seleccionadas.

El sistema debe volver a validar la disponibilidad al guardar, no solo al mostrarla.

Boceto elaborado para este proyecto. Las notas inferiores indican navegación y condiciones de uso.

Qué muestra y por qué

La fecha y los bloques de horario se presentan junto a la tarifa. El turno de 18:00 a 19:00 aparece ocupado porque R01 ya lo utiliza. El turno de 19:00 a 20:00 puede ofrecerse: el final de un intervalo no ocupa el inicio del siguiente.

Cómo se usaría

Reservar transfiere la selección al formulario. Al guardar se revisa otra vez la disponibilidad, porque mostrar un turno libre no garantiza que siga libre. La versión con varios usuarios requerirá una operación atómica en el servidor.

Requerimientos relacionados: RF-04, RF-05 y RF-06.

Formulario de reserva



Boceto elaborado para este proyecto. Las notas inferiores indican navegación y condiciones de uso.

Qué muestra y por qué

Se reúnen cliente, cancha, fecha y horas antes de guardar. El total se presenta para que recepción pueda comunicarlo sin hacer una cuenta aparte. En el ejemplo, una hora de Cancha Norte cuesta 18 000 CRC. La nueva reserva se inicia como pendiente.

Cómo se usaría

Guardar ejecuta las validaciones y vuelve al detalle cuando tiene éxito. Cancelar vuelve sin crear el registro. Si hay un cruce, se conserva lo escrito y se muestra un mensaje específico; borrar el formulario obligaría a repetir datos que sí eran correctos.

Requerimientos relacionados: RF-05, RF-06 y RF-10.

Ocho reglas de negocio explicadas

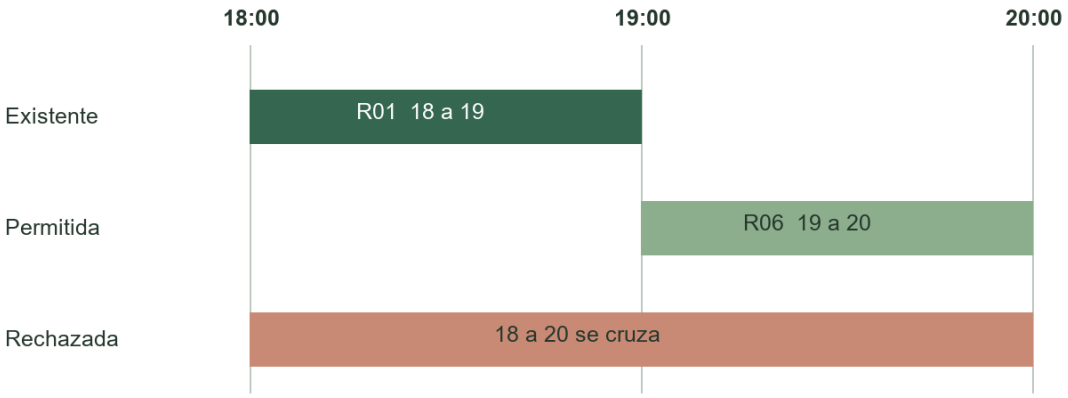
Regla	Condición y motivo
C1 Canchas activas	filter devuelve canchas activas del deporte solicitado. El mantenimiento conserva el registro, pero impide ofrecerlo.
C2 Horario válido	every exige horas enteras. El inicio debe ser menor que el fin y ambos deben quedar entre apertura y cierre.
CL1 Correo único	some busca un correo repetido después de quitar espacios y normalizar mayúsculas. También se revisa una forma básica de correo.
CL2 Límite diario	filter cuenta las reservas no canceladas de un cliente en una fecha. Debe estar activo y tener menos de tres para agregar otra.
R1 Disponibilidad	some detecta una reserva vigente de la misma cancha y fecha que se cruce. Las canceladas no bloquean el turno.
R2 Total	find ubica la cancha. La duración válida multiplicada por tarifaHora produce el total ordinario.
P1 Saldo	filter separa los pagos de la reserva y reduce suma los abonos. Saldo = total menos suma abonada.
P2 Pago válido	Rechaza monto no entero o no positivo, duplicado, sobrepago, reserva cancelada o referencia SINPE ausente o repetida.

validarReserva reúne las comprobaciones de relaciones, fecha, horario, cliente y disponibilidad antes de agregar. Repositorio es deliberadamente genérico y no conoce estas reglas; llamar agregar por sí solo no equivale a validar una reserva.

Las validaciones de este prototipo no son una barrera de seguridad frente a clientes externos. Cuando exista una API, el servidor deberá validar de nuevo y gestionar las solicitudes simultáneas.

Cómo comprobar los cruces de horario

Figura 5 Horarios que se cruzan y horarios consecutivos



Regla: nuevoInicio < finExistente y nuevoFin > inicioExistente.

El final de una reserva puede coincidir con el inicio de otra. Las barras representan intervalos del mismo día y de la misma cancha.

```
nueva.inicio < existente.fin &&
nueva.fin > existente.inicio
```

Las dos comparaciones deben ser verdaderas para que haya un cruce. Si la existente ocupa 18-19 y la nueva 19-20, la primera comparación es 19 < 19, que es falsa. Por eso el turno consecutivo se acepta. Si la nueva ocupa 18-20, ambas condiciones son verdaderas y se rechaza.

También se comparan canchaId y fecha. Dos canchas diferentes pueden tener reservas a la misma hora. Una reserva cancelada se ignora al revisar disponibilidad. El ejemplo usa horas enteras; no admite fracciones ni turnos que crucen la medianoche.

Pruebas de borde

Caso	Resultado esperado
18-19 y 19-20 en C01	Aceptar el segundo turno.
18-19 y 18-20 en C01	Rechazar el cruce.
18-19 cancelada y una nueva 18-19	Aceptar si las demás reglas se cumplen.
21-23 con cierre a las 22	Rechazar por horario.
Fecha 2026-02-30	Rechazar aunque tenga apariencia de fecha.

Evidencia de ejecución del prototipo

Transcripción de una ejecución real. true significa que la condición se cumple; false indica que el caso fue rechazado o que no está disponible. Los dos rechazos ilustrados son resultados esperados.

```
CANCHA CLARA | Prototipo del Avance 1
Datos: 5 canchas, 5 clientes, 5 reservas, 5 pagos.
C1. Canchas activas de fútbol: Cancha Norte, Cancha Sur
C2. Horario 18-19 válido: true
CL1. Correo ANA@example.com disponible: false
CL2. Ana puede reservar: true
R1. Cancha Norte 19-20 disponible: true
R2. Total de R01 en CRC: 18000
P1. Saldo R03 en CRC: 5000
P2. Abono de 6000 para R03 válido: false
Repositorio: registros tras agregar: 6
Repositorio: buscar R06: pendiente
Repositorio: pendientes: 4
Repositorio: eliminar R06: true
Polimorfismo: cotizaciones de 2 horas en CRC: 36000 / 32400
Caso controlado: La cancha ya está reservada en ese horario.
Fin: las ocho reglas fueron demostradas.
```

Cómo interpretar los resultados

El correo de Ana no está disponible porque ya existe, aunque se escriba con mayúsculas. La reserva nueva de 19 a 20 sí cabe después de R01. R03 cuesta 10 000 CRC y ya tiene un abono de 5 000, de modo que rechazar otro de 6 000 evita cobrar de más.

El repositorio pasa de cinco a seis registros al agregar R06. La búsqueda devuelve su estado y el filtro cuenta cuatro pendientes. Eliminar R06 devuelve true. Las cotizaciones 36 000 y 32 400 muestran que las dos clases responden de distinta manera a calcular(2).

El último caso intenta ocupar un turno ya reservado. El error se captura y se imprime; la ejecución llega al mensaje final. Esta prueba negativa deliberada demuestra que la regla se aplica sin detener toda la demostración.

Pruebas e incidencia real de instalación

Se ejecutaron seis pruebas automatizadas con node:test. Agrupan cruces y cancelaciones, horarios, clientes, relaciones y fechas, pagos y repositorio. El resultado fue seis aprobadas y cero fallidas; la transcripción se conserva en docs/evidencia/pruebas.txt.

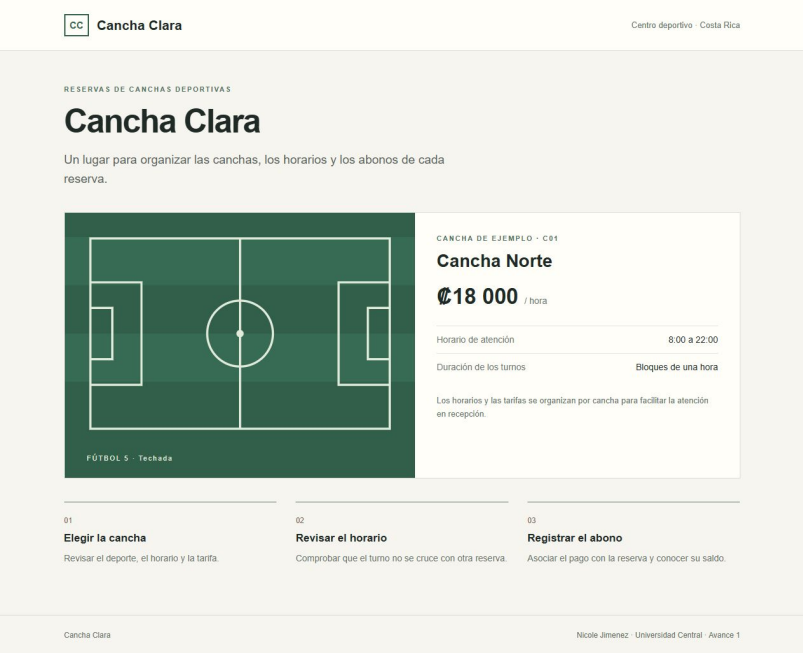
Incidencia observada

La primera creación de Angular generó los archivos, pero la instalación de paquetes terminó con EPERM al intentar crear una carpeta dentro de node_modules/@angular. El sistema operativo rechazó esa operación. No se comprobó si la causa exacta era un bloqueo temporal o los permisos del entorno.

```
npm error code EPERM
npm error syscall mkdir
.../frontend/node_modules/@angular
```

Se conservó el código generado y se repitió npm install desde frontend con una caché local dentro del espacio de trabajo. La instalación terminó correctamente. Después se generaron los tres componentes con CLI, se ejecutó npm run build y se abrió ng serve en el navegador.

Qué permite concluir. Fue una incidencia de instalación, no una regla de reservas defectuosa. La comprobación útil es que después el frontend compiló y se renderizó. No hace falta reproducir ese fallo para usar el proyecto, ni ejecutar npm como administrador como primer recurso.



Captura real posterior a la instalación. Se verificaron el nombre, la cancha, la tarifa y el horario; el navegador no registró errores de aplicación durante esa revisión.

Explicación del código y límites actuales

Por qué una interfaz no valida una reserva

Una interfaz describe qué campos deberían existir y sus tipos. Una Reserva con canchaId inexistente puede cumplir ese contrato porque el campo sigue siendo string. Por eso validarReserva busca el cliente y la cancha antes de comprobar disponibilidad.

Por qué buscar puede devolver undefined

find no siempre encuentra un registro. El tipo T | undefined obliga a comprobarlo. En la demostración se usa ?. para consultar un campo sin fallar; cuando la ausencia impide una operación, se lanza un Error con un mensaje concreto.

Qué se conserva y qué se reinicia

Los registros se guardan en arreglos. Cerrar el programa pierde cualquier cambio. Los cinco registros por entidad se cargan otra vez al iniciar. El frontend de este avance no está conectado con ese arreglo ni con una API: muestra una cancha de ejemplo con el mismo contrato.

Qué demuestra el getter de Angular

tarifaFormateada transforma tarifaHora en una cadena de moneda con Intl.NumberFormat. horario combina apertura y cierre. La plantilla consulta esos getters mediante interpolación. No se usa un pipe para esta tarea porque el avance pide explícitamente un getter.

Cómo continuar el proyecto

El siguiente paso es incorporar rutas, listas, formularios y datos por HTTP. Al guardar una reserva, se deben ejecutar las reglas en el servidor. La persistencia tendrá que conservar la tarifa aplicada a una reserva histórica para que cambiar el precio de una cancha no altere cobros antiguos. En este prototipo las tarifas permanecen fijas durante la ejecución.

Qué revisar antes de presentar

Ejecutar ambos programas, localizar cada requisito en su archivo y explicar con los datos de ejemplo por qué se acepta o rechaza una operación. Los commits locales registran el trabajo de esta preparación con su fecha real; no representan semanas de desarrollo previas. Los bocetos son propuestas, no capturas de funciones terminadas.

Repositorio: https://github.com/nidejimenezs/cancha-clara-ecn24